

编程基础 IV — 程序的结构

软件工程与计算 I · 2026 · 南京大学

| 刘钦

Reference

- 《计算机程序的构造和解释》 Harold Abelson, Gerald Jay Sussman
- 《代码大全》 Steve McConnell
- 《算法 + 数据结构 = 程序》 Niklaus Wirth (1976)
- 《Clean Code》 Robert C. Martin

知识点

- 代码是用来读的
- 降低复杂度的方法论
 - 分解与抽象
 - 层次性
- 程序 = 算法 + 数据结构
- 算法建模（基本表达式、组合、抽象）
- 迭代与递归
- 数据建模（基础数据、组合、抽象）
- 模块化
- 求值的环境模型
- 数据处理（输入、处理、输出）
- 日志

Outline

#	主题	分类
1	代码是用来读的	实践经验
2	降低复杂度 — 分解与抽象	方法学
3	编程 = 数据结构 + 算法	理论逻辑
4	算法建模（三种机制 + 迭代递归）	理论逻辑
5	数据建模	理论逻辑
6	模块化 + 求值的环境模型	理论逻辑
7	编程典型场景 — 数据处理	理论逻辑
8	用日志记录数据	实践经验
★	【AI视角】数据抽象 · 闭包 × AI Agent	AI连接

1. 代码是用来读的

实践经验

代码是用来读的

"Write once, Read many times." — Kent Beck

为什么可读性重要？

- **团队的需要：**代码需要被团队成员理解和维护
- **维护的需要：**代码生命周期中大部分时间在维护阶段

命名的重要性

糟糕的变量名字

```
int x;  
String xxx;  
double aretha;
```

名字不传递任何信息，读者无法理解代码意图。

良好的变量名字

```
int balance;  
int monthlyTotal;  
String customerID;
```

名字本身就是文档，读者无需注释即可理解。

2. 降低复杂度

分解 · 抽象 · 层次性

降低复杂度 — 方法一：分解（组合）

将复杂问题拆解为多个简单问题，再组合解决。

从最简单的元素出发：

```
print("*")      print("#")
```

组合成更复杂的操作：

```
* #  
// 打印 * #  
//= 打印一颗* + 打印一颗#
```

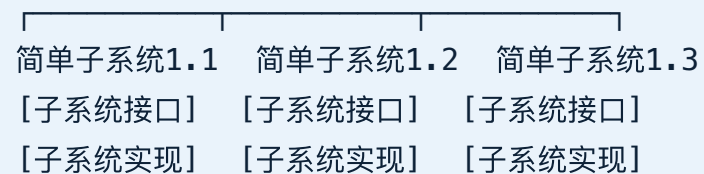
继续组合 → 打印 100 对 `*#`：

```
重复 100 次以下操作：{ print("*")    print("#") }
```

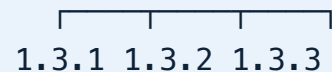
分解（组合）的关键点

复杂系统

↓ 分而治之（层次一）



↓ 分而治之（层次二）



- 分解之后每一部分**复杂度要变小**
- 相互之间**关联要小**，相对独立

降低复杂度 — 方法二：抽象 (Abstraction)

对重复模式命名和封装：接口的复杂度变小，接口和实现之间达成契约。

复杂系统 → (抽象) → 系统接口 / 系统实现

抽象示例 (Python)

```
def printSingleStarSharp():  
    print("*")  
    print("#")  
  
def printStars(n):  
    count = 0  
    while (count < n):  
        print("*")  
        count = count + 1
```

抽象的关键点：接口复杂度小；接口与实现之间达成契约

降低复杂度 — 方法三：层次性 (Hierarchy)

复杂系统通过层次结构组织：

自然界的层次性

- 科赫雪花（分形结构）
- 套娃（嵌套结构）
- 军队组织结构（树形层次）

软件中的层次性

应用层
业务逻辑层
数据访问层
数据库

分解 + 抽象 + 层次性 = 有效管理复杂度

3. Program = Algorithm + Data Structure

理论逻辑

Algorithms + Data Structures = Programs

Niklaus Wirth, 1976

"Algorithms + Data Structures = Programs" 是一本覆盖计算机编程基本议题的著作，尤其强调：算法和数据结构是内在相关的。

Wirth 于 1984 年获得图灵奖，以表彰其开发了一系列创新的计算机语言。

- 算法是计划、过程、步骤
- 数据结构是操作的目标、对象
- 算法和数据结构是配合的！

简单案例 — 线性表

数据结构	特点	访问	插入/删除
数组	连续内存	$O(1)$	$O(n)$
链表	指针连接	$O(n)$	$O(1)$
队列	FIFO	—	—
栈	LIFO	—	—

思考题：搜索线性表是否包含某个元素，用链表合适吗？

数组插入

有序场景

1. 遍历数组，找到插入位置
2. 将后续元素**后移一位**
3. 将新元素插入指定位置

[1, 2, _, 4, 5] → 插入 3

无规律场景

1. 假设插入下标位置是 1
2. 将目标位置元素**移到末尾**
3. 将新元素插入目标位置

[8, _, 7, 6, 2] → 插入 3

链表插入/删除

插入 x 结点 (在 b 和 c 之间):

→ a → b —————→ c .

 x 

(b.next = x, x.next = c)

删除 b 结点:

→ a → b → c .



(a.next = c, 跳过 b)

链表的插入/删除只需修改指针，**O(1)** 时间复杂度。

4. 算法建模

三种机制 · 迭代与递归

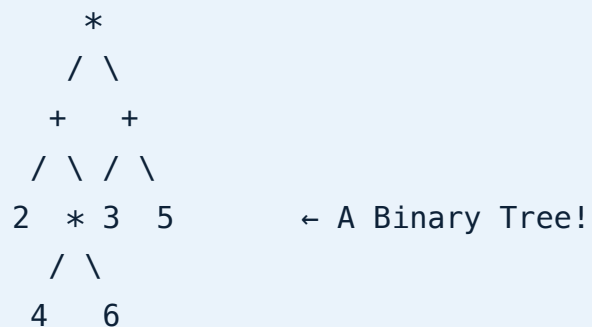
每种语言都会提供的三种机制

机制	说明	示例
基本表达式	语言提供的最简单单元	114 × 514 , True & False
分解（组合）	将简单元素组合成复杂元素	函数调用、表达式嵌套
抽象	将复合元素命名，作为单元操作	define 、函数定义

基本表达式 · 分解（组合）

分解（组合）— 树形表示法：

$(* (+ 2 (* 4 6)) (+ 3 5))$



每个表达式都可以用树来表示，树的求值是**自底向上**的。

抽象 — 给复合运算命名

```
; 定义平方: x 的平方
(define (square x) (* x x))

; 定义平方和: x 的平方 + y 的平方
(define (sum-of-squares x y)
  (+ (square x) (square y)))
```

参数与变量 (Python 示例):

```
def printStars(n):    # n 是形参
    count = 0         # count 是局部变量
    while (count < n):
        print("*")
        count = count + 1

printStars(9)         # 9 是实参
```

算法建模 — 两种思路

迭代 (Iteration) vs 递归 (Recursion)

以计算 n 的阶乘为例：

递归

```
(define (factorial n)
  (if (= n 1)
      1
      (* n (factorial (- n 1)))))
```

先展开，再折叠

需要记住所有中间状态

迭代

```
(define (factorial n)
  (fact-iter 1 1 n))

(define (fact-iter product counter max)
  (if (> counter max)
      product
      (fact-iter (* counter product)
                  (+ counter 1) max)))
```

每步只需当前状态，线性推进

计算 6! 的过程对比

线性递归过程

```
(factorial 6)
(* 6 (factorial 5))
(* 6 (* 5 (factorial 4)))
(* 6 (* 5 (* 4 (factorial 3))))
...
(* 6 (* 5 (* 4 (* 3 (* 2 1)))))
(* 6 (* 5 (* 4 (* 3 2))))
(* 6 (* 5 24))
(* 6 120)
720
```

线性迭代过程

```
(factorial 6)
(fact-iter 1 1 6)
(fact-iter 1 2 6)
(fact-iter 2 3 6)
(fact-iter 6 4 6)
(fact-iter 24 5 6)
(fact-iter 120 6 6)
(fact-iter 720 7 6)
720
```

线性递归 vs 线性迭代过程

递归写法和递归过程不是同一件事。

线性递归过程（堆栈增长）

```
(factorial 4)
(* 4 (factorial 3))      ← 挂起
(* 4 (* 3 (factorial 2))) ← 挂起
(* 4 (* 3 (* 2 1)))      ← 开始收缩
(* 4 (* 3 2))
(* 4 6)
24
```

每层调用都需要记住"还差什么", **$O(n)$ 栈空间**。

线性迭代过程（尾递归）

```
(factorial 4)
(fact-iter 1 1 4)
(fact-iter 1 2 4)
(fact-iter 2 3 4)
(fact-iter 6 4 4)
(fact-iter 24 5 4)
24
```

每步只需当前状态, **$O(1)$ 栈空间**。 `fact-iter` 是尾递归——递归调用是函数体的最后一步。

尾调用优化 (TCO): Scheme/Haskell 等语言保证尾递归不增长栈。Python / Java 不做 TCO, 深度尾递归会栈溢出。

尾调用优化 (Tail Call Optimization)

尾调用：递归调用是函数体的**最后一步**，调用者不需要保留任何状态。

Scheme — 保证 TCO

```
(define (fact-iter acc n)
  (if (= n 0)
      acc
      (fact-iter (* acc n) (- n 1))))

(fact-iter 1 1000000) ; ✓ 0(1) 栈
```

编译器将尾调用**复用当前栈帧**，等价于循环。

Python / Java — 无 TCO

```
def fact_iter(acc, n):
    if n == 0:
        return acc
    return fact_iter(acc * n, n - 1) # 尾调用

fact_iter(1, 1000) # ✓ 可以
fact_iter(1, 10000) # ✗ RecursionError
```

Python 硬限制默认调用栈深度 ≈ 1000 。即使是尾调用，每次调用仍压栈。

尾调用优化 — 语言对比

语言	TCO 保证	深递归行为
Scheme / Racket	✓ R7RS 强制要求	O(1) 栈, 等价循环
Haskell	✓ 惰性求值天然避免	安全
Python	✗ Guido 刻意不加	<code>RecursionError</code>
Java	✗ (JVM 限制)	<code>StackOverflowError</code>

为什么 Python 不加 TCO?

Guido van Rossum 认为栈帧对调试至关重要——TCO 会让 traceback 丢失调用历史, 难以排查错误。

在不支持 TCO 的语言中, 将尾递归手动改写为循环是标准做法。

案例 — 牛顿法求平方根

做什么? $y = \sqrt{x}$, 即找到 t 使得 $t^2 \approx x$

怎么做? 逐步逼近: 猜测 $\rightarrow$ 改进 $\rightarrow$ 直到足够精确

猜测	商	平均值 (改进后)
1	$2/1 = 2$	$(2+1)/2 = 1.5$
1.5	$2/1.5 = 1.3333$	$(1.3333+1.5)/2 = 1.4167$
1.4167	$2/1.4167 = 1.4118$	$(1.4167+1.4118)/2 = 1.4142$
1.4142	...	...

牛顿法 — 层次化分解

步骤分解（层次结构）：

```
sqrt
├─ sqrt-iter (迭代主体)
│   ├─ good-enough? (判断是否足够精确)
│   │   ├─ square
│   │   └─ abs
│   └─ improve (改进猜测)
│       └─ average
```

牛顿法 — Scheme 实现

```
(define (sqrt-iter guess x)
  (if (good-enough? guess x)
      guess
      (sqrt-iter (improve guess x) x)))

(define (improve guess x)
  (average guess (/ x guess)))

(define (average x y) (/ (+ x y) 2))

(define (good-enough? guess x)
  (< (abs (- (square guess) x)) 0.001))

(define (sqrt x) (sqrt-iter 1.0 x))
```

牛顿法 — 多语言版本

C 语言版

```
#include <math.h> /* fabs */
double mysqrt(float x) {
    double g0, g1;
    if (x == 0) return 0;
    g0 = x / 2;
    g1 = (g0 + x/g0) / 2;
    while (fabs(g1-g0) > 0.01) {
        g0 = g1;
        g1 = (g0 + (x/g0)) / 2;
    }
    return g1;
}
```

Python 版

```
c = float(input()) # input() 返回 str, 需转为 float
err = 1e-15
t = c
while abs(t - c/t) > err:
    t = (c/t + t) / 2.0
print(t)
```

斐波那契数列 — 迭代 vs 递归

迭代 (C++)

```
long fab_iteration(int index) {
    long f1=1L, f2=1L, f3=0;
    if (index==1||index==2)
        return 1;
    for(int i=0; i<index-2; i++) {
        f3 = f1 + f2;
        f1 = f2;
        f2 = f3;
    }
    return f3;
}
```

像《明日边缘》：不断循环推进

递归 (C++)

```
long fab_recursion(int index) {
    if(index==1||index==2)
        return 1;
    return fab_recursion(index-1)
        + fab_recursion(index-2);
}
```

像《盗梦空间》：层层嵌套深入

方法	时间复杂度	空间复杂度
迭代	$O(n)$	$O(1)$
递归	$O(\phi^n) \approx O(1.618^n)$	$O(n)$ 栈深

递归树大量重复计算（fib(n-2) 被算了多次），指数级增长。可用 记忆化 (memoization) 将递归降至 $O(n)$ 。

思考题：加法可不可以用递归和迭代表示？

5. 数据建模

基础数据 · 组合 · 抽象

数据建模 vs 算法建模

同一问题，两种视角：

基于算法建模

```
print("*")  
print("#")
```

用两步操作来描述问题

基于数据建模

```
print("*#")
```

用一个数据单元来描述问题

数据建模的三层次：基础数据 → 数据的组合 → 数据的抽象

基础数据 · 数据的组合

基础数据

- 整数 Integer
- 浮点数 Float, Double
- 布尔值 Boolean

数据的组合

方式	说明
数组	同一类数据的组合
结构体 struct	不同数据的组合
对象	数据和行为的组合

数据的抽象 — 有序对

有序对 (Ordered Pair) 是数据抽象的基础:

定义方式	公式
Wiener's	$(a, b) := \{\{\{a\}, \emptyset\}, \{\{b\}\}\}$
Hausdorff's	$(a, b) := \{\{a, 1\}, \{b, 2\}\}$
Kuratowski	$(a, b)_K := \{\{a\}, \{a, b\}\}$

思考题: 如何用有序对表达有理数? 如何表达树形结构数据?

有序对的应用 — 有理数

```
(define (make-rat n d) (cons n d)) ; 构造器
(define (numer x) (car x))         ; 选择器: 分子
(define (denom x) (cdr x))         ; 选择器: 分母

(define (add-rat x y)
  (make-rat (+ (* (numer x) (denom y))
               (* (numer y) (denom x)))
            (* (denom x) (denom y))))

(define (equal-rat? x y)
  (= (* (numer x) (denom y))
     (* (numer y) (denom x))))
```

抽象屏障：使用者只需知道接口 (`make-rat`, `numer`, `denom`)，无需关心实现。

改进实现（加入约分）——接口完全不变：

； 改进版：构造时自动约分，调用方代码零修改

```
(define (make-rat n d)
  (let ((g (gcd n d)))
    (cons (/ n g) (/ d g))))
```

； (make-rat 4 6) → (2 . 3)，而不是 (4 . 6)

这正是抽象的威力：实现可以悄悄升级，调用方无感知。

有序对的应用 — 链表与树

链表 (box-and-pointer)

→ [•|•] → [•|•] → [•|•] → [•|/]
↓ ↓ ↓ ↓
1 2 3 4

```
(list 1 2 3 4)
```

链表操作

```
(define (length items)
  (if (null? items) 0
      (+ 1 (length (cdr items)))))
```

树 (嵌套有序对)

((1 2) 3 4)
•
/ | \
(1 2) 3 4
/ \
1 2

树操作

```
(define (count-leaves x)
  (cond ((null? x) 0)
        ((not (pair? x)) 1)
        (else (+ (count-leaves (car x))
                   (count-leaves (cdr x))))))
```

6. 模块化

求值的环境模型

模块

模块内部

- **定义**：一段代码
- **对象**：独立的运行环境
- 维护局部状态变量

模块间交互

- **接口**：方法的参数和返回值
- **求值环境**：全局变量、共享变量
- **接口说明**：使用时需要知道的信息

求值的环境模型 — 创建过程

```
(define (square x) (* x x))  
; 等价于  
(define square (lambda (x) (* x x)))
```

执行后, `square` 在**全局环境**中绑定到一个过程对象:

```
global env → [ other variables ... ]  
               square: → [•|•]  
                        ↓  
                parameters: x  
                body: (* x x)
```

求值的环境模型 — 应用过程

```
(square 5)
```

调用时，创建新环境 **E1**，其中 `x: 5`，在 E1 中对 `(* x x)` 求值：

```
global env → [ square: ... ]  
              ↑  
E1 → [ x: 5 ]  
      (* x x)
```

无赋值时：环境之间无约束，完全通过参数传递数值。

有赋值过程 — 局部状态

```
(define (make-withdraw balance)
  (lambda (amount)
    (if (>= balance amount)
        (begin (set! balance (- balance amount))
                balance)
        "Insufficient funds"))))

(define W1 (make-withdraw 100))
(define W2 (make-withdraw 100))
```

W1 和 W2 各自维护独立的 balance:

```
global env → [ W1: ... ][ W2: ... ]
               E1: balance=50   E2: balance=100
```

E1、E2 是不同的环境，不同的空间，不同的影响范围。
这正是面向对象中 **this** 指针的本质！

内部定义 — 环境模型的延伸

函数内部可以用 `define` 定义局部子程序，它们只活在该函数的环境中。

```
(define (sqrt x)
  (define (good-enough? guess)
    (< (abs (- (square guess) x)) 0.001))
  (define (improve guess)
    (average guess (/ x guess)))
  (define (sqrt-iter guess)
    (if (good-enough? guess)
        guess
        (sqrt-iter (improve guess))))
  (sqrt-iter 1.0))
```

内部定义创建局部环境

```
global env → [ sqrt: ... ]  
           ↑  
E_sqrt → [ x: 2, good-enough?: ...,  
          improve: ..., sqrt-iter: ... ]
```

`good-enough?`、`improve` 对外不可见。

信息隐藏 (Information Hiding)

- 外部只需知道 `sqrt`，不需要知道 `improve`
- 名字不污染全局命名空间
- 内部实现可以任意修改

这正是**模块化**的实现机制：接口 vs 实现的边界。

7. 编程的典型场景

数据处理：输入 → 处理 → 输出

数据处理模型

所有程序都可以用 输入 → 处理 → 输出 来描述：

输入 (Input) → 处理 (Process) → 输出 (Output)

案例：求两个数之和

阶段	操作
输入	从控制台读入 2 个数（字符串）
处理	将 String 转换为 int；计算求和
输出	打印结果

8. 用日志记录数据

实践经验

日志使用场景

你想要执行的任务	最好的工具
结果显示在控制台	<code>print()</code>
普通操作事件报告（状态监控、错误调查）	<code>logging.info()</code> / <code>logging.debug()</code>
提出警告信息	<code>warnings.warn()</code> / <code>logging.warning()</code>
报告运行时错误	引发异常
报告错误但不引发异常	<code>logging.error()</code> / <code>logging.critical()</code>

日志级别

级别	何时使用
DEBUG	细节信息，仅当诊断问题时适用
INFO	确认程序按预期运行
WARNING	有已经或即将发生的意外（程序仍可运行）
ERROR	严重问题，程序某些功能不能正常执行
CRITICAL	严重错误，程序已不能继续执行

日志示例

```
import logging
logging.warning('Watch out!') # 会打印到控制台
logging.info('I told you so') # 不会打印（低于默认级别）
```

输出: `WARNING:root:Watch out!`

多模块日志:

```
# myapp.py
import logging, mylib

def main():
    logging.basicConfig(filename='myapp.log', level=logging.INFO)
    logging.info('Started')
    mylib.do_something()
    logging.info('Finished')

# mylib.py
import logging
def do_something():
    logging.info('Doing something')
```

myapp.log 内容: INFO:root:Started / INFO:root:Doing something / INFO:root:Finished

本讲总结

方法学

1. 代码可读性是软件工程基础
2. 分解 + 抽象 + 层次性 = 降低复杂度
3. 迭代 vs 递归：两种算法思路

理论逻辑

4. 程序 = 算法 + 数据结构
5. 三种机制：基本表达式 / 分解 / 抽象
6. 数据建模：基础 → 组合 → 抽象（有序对）
7. 环境模型：理解变量作用域与状态
8. 数据处理：输入 → 处理 → 输出

【AI视角】

- 数据抽象 ↔ LLM 知识封装：构造器/选择器/屏障 = 参数/Prompt/分层接口
- 闭包与状态 ↔ AI Agent 记忆：短期（上下文）+ 长期（向量库）+ 工具调用

核心洞见

算法和数据结构是配合的！

— Niklaus Wirth

分解、抽象、层次性——这三件事合在一起，就是人类管理复杂度的全部秘密。

环境之间有约束，新环境维护局部状态变量——这就是对象的本质。

课后思考

1. 搜索线性表是否包含某个元素，用链表合适吗？为什么？
2. 加法可不可以用递归和迭代两种方式来表示？请写出代码。
3. 用有序对表示一棵二叉搜索树，应该如何设计数据结构？
4. `(define W1 (make-withdraw 100))` 和 `(define W2 (make-withdraw 100))` 各自的 `balance` 是独立的吗？为什么？

【AI视角】数据抽象 × LLM 的"知识封装"

程序中的数据抽象

- **构造器**：创建对象，隐藏内部表示
- **选择器**：通过接口访问数据
- **抽象屏障**：上层代码不依赖下层实现

； 点的两种实现对上层透明

```
(define (make-point x y) (cons x y))
```

```
(define (x-coord p) (car p))
```

LLM 的"知识封装"

- **参数权重** ↔ 构造器（训练时编码知识）
- **Prompt 接口** ↔ 选择器（查询知识）
- **预训练/微调分层** ↔ 抽象屏障

用户只需关心：

输入什么 Prompt → 得到什么输出

不必了解：

权重矩阵如何存储、前向传播细节

好的抽象设计让使用者无需了解实现细节——无论是 `make-point`，还是 GPT 的 API。

【AI视角】闭包与状态 × AI Agent 的记忆

闭包维护局部状态

```
(define (make-counter)
  (let ((n 0))
    (lambda ()
      (set! n (+ n 1))
      n)))

(define c1 (make-counter))
(c1) ; 1
(c1) ; 2 ← 状态在闭包中持久化
```

闭包 = 函数 + 捕获的环境

AI Agent 的"记忆"机制

- **短期记忆**（上下文窗口） ↔ 函数调用栈
- **长期记忆**（向量数据库） ↔ 闭包持久状态
- **工具调用**（Function Calling） ↔ 高阶函数

```
# Agent 闭包：每次调用保持历史
agent = make_agent(memory=VectorDB())
agent("你好") # 第1轮
agent("继续上面的话题") # 第2轮，记得历史
```

理解闭包和环境，就理解了 AI Agent 如何在无状态 API 之上构建有记忆的对话。

编程基础 IV — 程序的结构

软件工程与计算 I · 2026 · 南京大学

感谢聆听